Tabla de contenidos

1. Terminal
2. Comandos
3. Ejecución de Python en Terminal

Terminal

1. ¿Qué es la terminal o consola?

Una terminal o consola es una de las formas para comunicarnos con un computador. Esta nos provee de una interfaz instrucciones o comandos al computador. Por ejemplo, ejecutar programas instalados previamente como Python y Jupyter Notebook, navegar por el sistema de archivos del computador, explorar qué archivos hay en determinadas carpetas, entre otros.

El funcionamiento de esta interfaz es a través de comando de línea de texto, es decir, su funcionamiento se basa en que el usuario escribe comandos junto con sus parámetros para que el sistema operativo entregue una respuesta. A continuación se presenta una foto de una terminal en MAC donde se ejecutó el comando `cd` para acceder al escritorio del computador y `ls` para saber qué archivos y carpetas hay en el escritorio

Para efectos de este curso, las dos principales ventajas de utilizar la terminal son:

1. Poder tener varias terminales abiertas y cada una utilizarla para diferentes propósitos. Por ejemplo, **ejecutar diferentes archivos Python de forma simultánea**.

2. Ser capaz de saber siempre desde qué carpeta estamos ejecutando un archivo. Lo cual nos reducirá la posibilidad de tener errores relacionados a **rutas relativas**.

2. ¿Cómo abrir la terminal?

- **Windows**: En la barra de inicio, puedes buscar "cmd" y abrir el programa ("Command prompt"). Aunque también recomendamos descargar desde la tienda la aplicación "Windows Terminal" y usarla como terminal.
- **Linux**: Puedes usar la combinación de teclas `Ctrl + Alt + T` para abrir la terminal.
- **macOS**: En Finder, abre la carpeta /Aplicaciones/Utilidades y haz doble clic en "Terminal".

Comandos

Los comandos corresponden al mecanismo donde el usuario se puede comunicar con el computador y solicitar que haga ciertas acciones. Para este curso, los comandos más importantes a conocer son 6: `pwd`, `dir` o `ls`, `cd`, `mkdir`, `echo` o `touch` y `python`.

1. Formato del comando

El formato para todo comando es el siguiente:

`comando argumentos* -opciones*`

- **`comando`**: corresponde al nombre del comando a llamar. Por ejemplo, `pwd`, `cd`, `python`, `pip`.
- **`argumentos*`**: corresponde a los parámetros que el comando va a utilizar para su ejecución. Dependiente del parámetro, estos argumentos pueden ser 0, 1, 2, etc. Algunos casos son:
    a. **`pwd`** es un comando que no necesita argumentos, solo escribimos dicho comando y funcionará.
    b. **`python`** es un comando que cambia su funcionamiento según la cantidad de argumentos. Si se ejecuta sin ningún argumento (`python`), se abrirá la consola de Python. Si se ejecuta con 1 argumento que corresponde a un archivo .py (`python ejemplo3.py`), se ejecutará el archivo correspondiente.
    c. **`pip`** es un comando que interactúa con el gestor de librerías en Python, y este tiene muchos argumentos posibles. Para ver las librerías instaladas, se necesita de 1 argumento, `list`, para su ejecución: `pip list`. Para instalar una librería, se necesita 2 argumentos, `install nombre_librería`, para su ejecución: `pip install pyqt5`.
- **`-opciones`**: también conocidos como *flags*, es la forma de especificar opciones durante la ejecución del comando. Cada opción se escribe como `-nombre_de_la_opción parámetro*` en donde el parámetro puede ser opcional según la opción que estemos llamando. Por ejemplo:
    a. Para instalar una librería de python, se puede hacer `pip install pyqt5`, pero también se puede hacer `pip install pyqt5 -q`. El `-q` se encarga de personalizar la ejecución, en este caso, hacer la instalación "quiet", es decir, sin imprimir tantas cosas en consola.

2. Uso de comandos

Para ejemplificar cada comando que veremos a continuación, imaginemos que tenemos la siguiente estructura de carpetas y archivos:

Windows

En Windows tenemos 4 comandos básicos: `cd`, `dir`, `mkdir` y `echo`.

1. Ver directorio actual (`cd`)

Del inglés **c**hange **d**irectory, este comando nos permite movernos en el sistema de archivos para acceder a diferentes directorios/carpetas. Cuando ejecutamos este comando sin ningún argumento, nos retorna la posición actual como una **ruta absoluta**.

```
→ cd
C:\Users\Pedro
```

En este caso, nuestra terminal está ubicada dentro de la carpeta `Pedro` que a su vez esta carpeta está dentro de la carpeta `Users`, la cual está dentro de la carpeta raíz.

2. Cambiar directorio actual (`cd`)

En caso de que se agregue una ruta (absoluta o relativa) al comando, este nos permitirá cambiar nuestro directorio.

```
→ cd
C:\Users\Pedro
→ cd Libros
→ cd
\Users\Pedro\Libros
```

Ahora, desde la carpeta `Pedro` nos movimos a la carpeta `Libros`. También es posible subir a la carpeta superior utilizando el argumento `..`.

```
→ cd
C:\Users\Pedro
→ cd Libros
→ cd
C:\Users\Pedro\Libros
→ cd ..
→ cd
C:\Users\Pedro
→ cd Música
→ cd
C:\Users\Pedro\Música
→ cd ..\..\data
→ cd
\data
```

3. Listar contenido de un directorio (`dir`)

Del inglés **dir**ectory, este comando nos entrega un detalle de los archivos y carpetas presentes en un determinado directorio. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Qué tenemos en la carpeta?

```
→ cd
\Users\Pedro
→ dir
... (texto omitido para este ejemplo)

Directorio de \Users\Pedro
02-08-2022  12:29    <DIR>          .
02-08-2022  12:29    <DIR>          ..
19-07-2022  18:45    <DIR>          Fotos
19-07-2022  18:45    <DIR>          Libros
19-07-2022  18:45    <DIR>          Música
```

```
... (texto omitido para este ejemplo)
→ Libros
→ cd
\Users\Pedro\Libros
→ dir
... (texto omitido para este ejemplo)

Directorio de \Users\Pedro\Libros
02-08-2022  12:29    <DIR>          .
02-08-2022  12:29    <DIR>          ..
02-08-2022  12:29            11.960 calculo.pdf
02-08-2022  11:55            16.119 python.pdf
... (texto omitido para este ejemplo)
```

4. Crear un directorio nuevo (`mkdir`)

Del inglés **m**a**k**e **dir**ectory, este comando crea una nueva carpeta o subcarpeta con el nombre indicado como argumento. También podemos entregarle el *path* con la dirección hacia la nueva carpeta.

```
→ cd
\Users\Pedro
→ dir
... (texto omitido para este ejemplo)

Directorio de \Users\Pedro
02-08-2022  12:29    <DIR>          .
02-08-2022  12:29    <DIR>          ..
19-07-2022  18:45    <DIR>          Fotos
19-07-2022  18:45    <DIR>          Libros
19-07-2022  18:45    <DIR>          Música
... (texto omitido para este ejemplo)
```

Ahora, crearemos una nueva carpeta llamada Codigos. Esta carpeta no existe antes del comando y se crea en el momento en que lo ejecutamos.

```
→ mkdir Codigos
→ dir
... (texto omitido para este ejemplo)

Directorio de \Users\Pedro
02-08-2022  12:29    <DIR>          .
02-08-2022  12:29    <DIR>          ..
19-07-2022  18:45    <DIR>          Fotos
19-07-2022  18:45    <DIR>          Libros
19-07-2022  18:45    <DIR>          Música
07-03-2024  10:00    <DIR>          Codigos
... (texto omitido para este ejemplo)
```

5. Crear un archivo nuevo (`echo`)

Este comando permite mostrar un mensaje de salida en la terminal, lo mas parecido a un print de Python pero en consola. En este caso, lo usaremos para redirigir esa salida a un archivo. La sintaxis es: `echo "TEXTO" > NOMBRE_ARCHIVO`, donde `echo` es el nombre del comando, `"TEXTO"` es el texto de salida entre comillas simples o dobles, y `NOMBRE_ARCHIVO` será el nombre de nuestro nuevo archivo considerando siempre la extensión (.txt, .py).

Para crear un archivo vacío de Python, simplemente dejaremos el TEXTO como un string vacío `""`, y le daremos el nombre junto a la extensión .py.

```
→ echo "" > nuevo.py
```

¡Te invitamos a probar estos 4 comandos (`cd`, `dir`, `mkdir` y `echo`) en tu terminal para familiarizarte con su uso! Utilizarás mucho estos comandos en este curso.

Linux y macos

Tanto Linux como Mac OS X funcionan con facilidades estándar de Unix en sus terminales. Esto nos permite utilizar los mismos comandos para ambos sistemas operativos: `pwd`, `cd`, `ls`, `mkdir` y `touch`.

1. Ver directorio actual (`pwd`)

De las siglas en inglés **p**rint **w**orking **d**irectory, es el comando que nos retorna la **ruta absoluta** para llegar a nuestra posición desde la terminal. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Dónde estamos?. Cuando ejecutamos este comando, nos retorna la posición actual como una **ruta absoluta**.

```
→ pwd
/Users/Pedro
```

En este caso, nuestra terminal está ubicada dentro de la carpeta `Pedro` que a su vez esta carpeta está dentro de la carpeta `Users`, la cual está dentro de la carpeta raíz.

2. Cambiar directorio actual (`cd`)

Del inglés **c**hange **d**irectory, este comando nos permite movernos en el sistema de archivos para acceder a diferentes directorios/carpetas. Para esto, es necesario agregar una **ruta absoluta** o **relativa** como argumento.

```
→ pwd
/Users/Pedro
→ cd Libros
→ pwd
/Users/Pedro/Libros
```

Ahora, desde la carpeta `Pedro` nos movimos a la carpeta `Libros`. También es posible salir de una carpeta utilizando el argumento `..`.

```
→ pwd
/Users/Pedro
→ cd Libros
→ pwd
/Users/Pedro/Libros
→ cd ..
→ pwd
/Users/Pedro
→ cd Música
→ pwd
/Users/Pedro/Música
→ cd ../../data
→ pwd
/data
```

3. Listar contenido de un directorio (`ls`)

Del inglés **ls**t, este comando nos entrega un detalle de los archivos y carpetas presentes en un determinado directorio. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Qué tenemos en la carpeta?

```
→ pwd
/Users/Pedro
→ ls
Fotos
Libros
Música
→ Libros
→ cd
/Users/Pedro/Libros
→ ls
calculo.pdf
python.pdf
```

4. Crear un directorio (`mkdir`)

Del inglés **m**a**k**e **dir**ectory, este comando crea una nueva carpeta o subcarpeta con el nombre indicado como argumento. También podemos entregarle el *path* con la dirección hacia la nueva carpeta.

```
→ pwd
/Users/Pedro
→ ls
Fotos
Libros
Música
→ mkdir Codigos
→ ls
```

```
Codigos
Fotos
Libros
Música
→ mkdir /Users/Pedro/Apuntes
→ ls
Apuntes
Codigos
Fotos
Libros
Música
```

5. Cear un archivo vacío (`touch`)

Este comando "toca" el archivo para comprobar si existe o no en esa ruta. Si un archivo no existe en el directorio indicado, crea un archivo vacío con el nombre y la extensión. Si el archivo existe, modifica instantáneamente el *timestamp* o fecha de último acceso al archivo al momento de ejecución del comando. Su nombre "touch" en español es "tocar", como si estuvieras comprobando si está ahi o no con un palito. De forma informal, podemos decir que este comando nos responde a la pregunta: ¿Existes en este momento?.

```
→ touch main.py
```

Al ejecutar este comando, la terminal no emitirá ningún mensaje. Si el archivo pedido main.py no existe, podrás comprobar con `ls` que ha sido creado y está vacío.

¡Te invitamos a probar estos 5 comandos (`pwd`, `cd`, `ls`, `mkdir` y `touch`) en tu terminal para familiarizarte con su uso! Utilizarás mucho estos comandos en este curso.

Ejecución de Python en Terminal

La mayoría de las *IDEs* para programar en Python ofrecen opción de ejecutar esos archivos con solo oprimir un botón. No obstante, en muchos casos, su interfaz no muestra claramente desde donde se está ejecutando el archivo, es decir, no se sabe tan fácilmente el `pwd` desde donde se ejecutó el archivo Python. Esto en varias ocasiones genera **problemas** con la apertura de archivos y también `imports` o cuando utilizamos **rulas relativas** en el código.

Adicionalmente, estas *IDEs* vienen por defecto con la configuración para ejecutar **1 archivo a la vez**. Esto provoca que para actividades donde el estudiantado requiere ejecutar **2 o más archivos a la vez**, no se logre.

En vista de las dificultades presentadas anteriormente, a continuación se explica cómo poder ejecutar archivos de Python en terminal. Esto nos dará la ventaja de:

1. Saber claramente nuestra ubicación en el sistema de archivos.
2. Evitar problemas con la apertura de archivos, los `imports` y con las **rulas relativas** dentro del código.
3. Poder ejecutar más de 1 archivo simultáneamente.

1. Abrir Python

Desde la terminal, podemos escribir `python` o `python3` y podremos acceder a la consola de Python, donde podremos ejecutar comandos de python. Por ejemplo, si necesitamos hacer una operación rápida (y no queremos usar nuestra calculadora).

2. Ejecutar *script* de Python

Otra ventaja de usar la terminal es poder ejecutar nuestros archivos Python sin recurrir a alguna *IDE* (como Visual Studio Code o SublimeText3). Para esto, solo basta agregar como argumento el nombre del archivo a ejecutar.

En el siguiente caso, vamos a acceder a la carpeta `semana-01` que está dentro de `contenidos` en el escritorio y vamos a ejecutar el archivo `ejemplo.py`

También es posible ejecutar un archivo que no esté dentro de nuestro directorio actual. Para esto, es necesario escribir la **ruta absoluta** o **relativa** a dicho archivo. En el siguiente ejemplo se ejecutó el mismo archivo, pero en el primer caso se utilizó una **rula relativa** para acceder a él y en el segundo caso se utilizó una **ruta absoluta**.

⚠ **Importante** ⚠: esto hace que el `ejemplo.py` crea que está siendo ejecutado desde la ruta actual de la consola, que es distinta a la carpeta donde se encuentra el archivo. Lo cual puede generar un error.

3. Errores comunes en Python con la terminal

1. **Archivo no existe**

Es posible que el archivo a ejecutar o la ruta a dicho archivo no estén bien escritos o simplemente no exista dicho archivo. En ese caso Python retornará un error indicando que el archivo no existe.

1. **Conflictos con rutas relativas**

Es posible que el archivo a ejecutar ocupe rutas relativas y esto genere conflictos según donde ejecutamos el archivo. Por ejemplo, junto a estos contenidos hay un directorio llamado `carpeta_auxiliar`. Dentro de este existen 2 archivos: `hello_world.txt` y `main.py`. El contenido de este segundo archivo es

```
import os


if __name__ == '__main__':
    print("Me estoy ejecutando desde:")
    path = os.getcwd()
    print(path)
    content = open("hello world.txt").read()
    print(content)
```

Es decir, este nuevo archivo nos indica desde donde se está ejecutando el Python y luego abre `hello world.txt` para imprimir su contenido. Dado que solo está escrito el nombre del

archivo, `main.py` asume que `hello world.txt` está en el mismo directorio desde donde se ejecutará `main.py`.

Ahora, desde la terminal, vamos a posicionarnos en la carpeta de los contenidos de la semana 1 y vamos a intentar ejecutar el `main.py`.

Nos salió el siguiente error:

```
Traceback (most recent call last):
  File
"/Users/macbook/Desktop/contenidos/semana-01/carpeta_auxiliar/main.py"
, line 8, in <module>
    content = open("hello world.txt").read()
FileNotFoundError: [Errno 2] No such file or directory: 'hello
world.txt'
```

Esto ocurrió porque nosotros ejecutamos `main.py` desde el directorio `/Users/macbook/Desktop/contenidos/semana-01`. En dicho directorio **no existe `hello world.txt`**.

Ahora, utilizaremos `cd` para acceder al directorio `carpeta_auxiliar` y volvemos a ejecutar `main.py`.

Ahora, sí funciona porque efectivamente en `/Users/macbook/Desktop/contenidos/semana-01/carpeta_auxiliar` sí existe el archivo `hello world.txt`.

Importante: siempre tengan en mente desde qué carpeta se ejecutarán sus archivos Python para no tener problema con las rutas relativas.